

TABLE OF CONTENTS

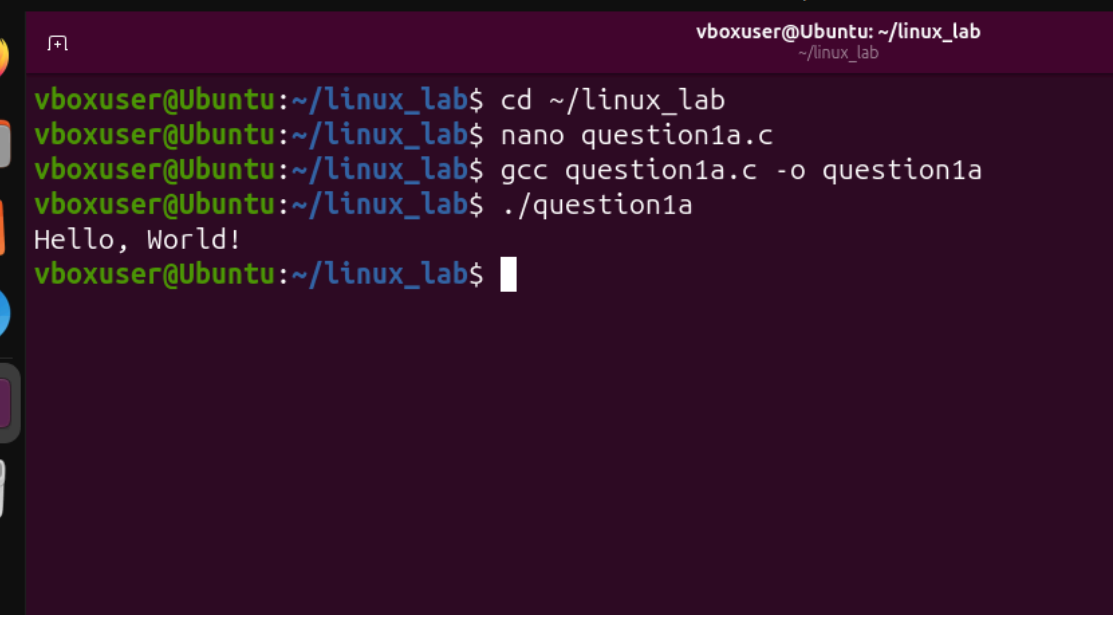
TABLE OF CONTENTS	1
II) C PROGRAM STRUCTURE AND EXAMPLES	3
III. GCC COMPILATION PROCESS	5
IV. RETURN VALUES AND EXIST STATUS	8
V. KNOWLEDGE CHECK QUESTIONS	9
B. Question 2	10
C. Question 3	10
D. Question 4	11
E. Question 5	11
F . Question 6	12
VI . LAB TASKS COMPLETED	12

LIST OF FIGURES

- Figure 1: Compilation and Execution of Basic Hello World C Program
- Figure 2: Compilation and Execution of C Program with User Input
- Figure 3: Compilation and Execution of C Program with Formatted Output
- Figure 4: GCC Installation and Version Verification on Linux
- Figure 5: Preprocessing Stage of C Compilation
- Figure 6: Preprocessing Stage of C Compilation
- Figure 7: Stage 2 – Compilation of C Program to Assembly
- Figure 8: Stage 3– Compilation of C Program to Assembly
- Figure 9: Stage 4 – Linking Object File to Create Executable
- Figure 10: C Program Execution and Return Value Verification
- Figure 11: Checking Program Exit Status Using echo \$?
- Figure 12: Task 1 – Program Execution with Return Value 0
- Figure 13: Task 2 – Program Execution with Return Value 1
- Figure 14: Task 3 – C Program Accepting and Displaying User Input
- Figure 15: Task 4 – Stage 1 Preprocessing and Generated Output File
- Figure 16: Task 4 – Stage 2 Compilation to Assembly
- Figure 17: Task 4 – Stage 3 Assembly to Object File
- Figure 18: Task 4 – Stage 4 Linking and Execution
- Figure 19: Task 5 – Program Execution with Return Value 50

II) C PROGRAM STRUCTURE AND EXAMPLES

A. Basic Hello World Program



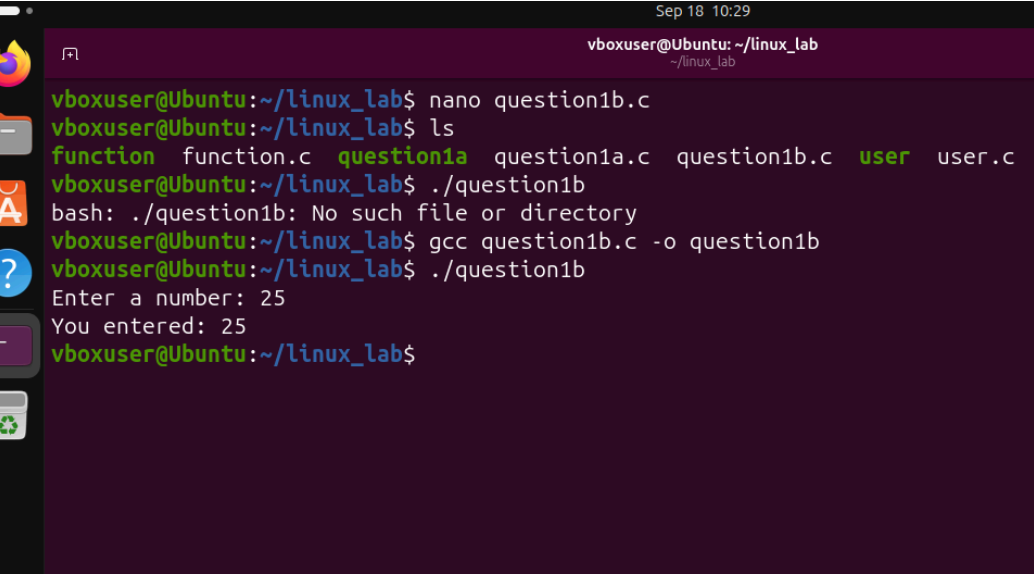
```
vboxuser@Ubuntu: ~/linux_lab
vboxuser@Ubuntu:~/linux_lab$ cd ~/linux_lab
vboxuser@Ubuntu:~/linux_lab$ nano question1a.c
vboxuser@Ubuntu:~/linux_lab$ gcc question1a.c -o question1a
vboxuser@Ubuntu:~/linux_lab$ ./question1a
Hello, World!
vboxuser@Ubuntu:~/linux_lab$
```

Figure 1: Compilation and Execution of Basic Hello World C Program

Summary:

Created and compiled a basic C program using GCC. The program was successfully executed from the linux_lab directory and displayed “Hello, World!” in the terminal.

B. Program with User Input



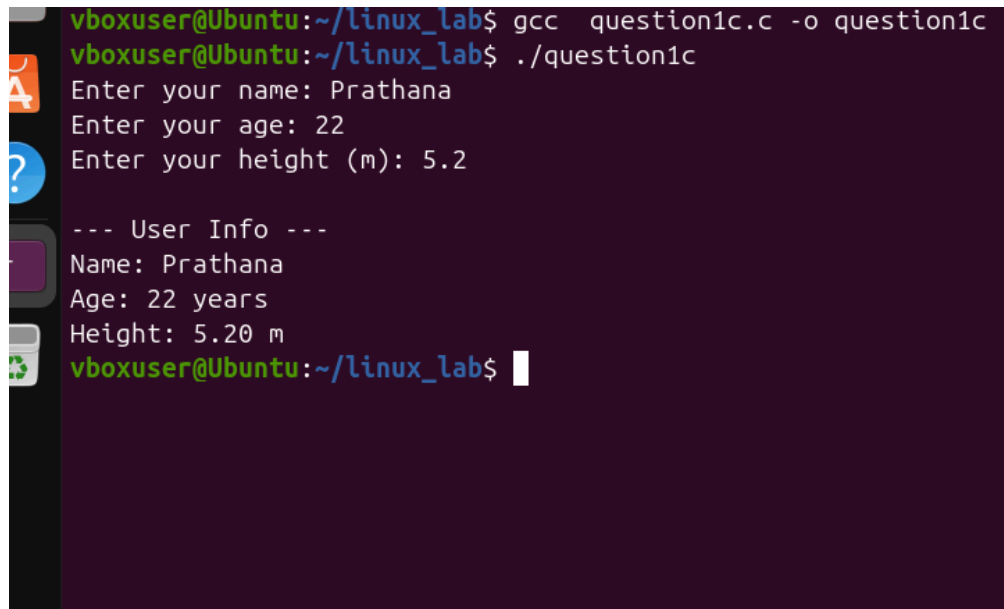
```
Sep 18 10:29
vboxuser@Ubuntu: ~/linux_lab
vboxuser@Ubuntu:~/linux_lab$ nano question1b.c
vboxuser@Ubuntu:~/linux_lab$ ls
function  function.c  question1a  question1a.c  question1b.c  user  user.c
vboxuser@Ubuntu:~/linux_lab$ ./question1b
bash: ./question1b: No such file or directory
vboxuser@Ubuntu:~/linux_lab$ gcc question1b.c -o question1b
vboxuser@Ubuntu:~/linux_lab$ ./question1b
Enter a number: 25
You entered: 25
vboxuser@Ubuntu:~/linux_lab$
```

Figure 2: Compilation and Execution of C Program with User Input

Summary:

Created a C program that accepts a number from the user, compiled it using GCC, and executed it successfully. The program accepted the input 25 and displayed the corresponding output "You entered: 25".

C. Program with Formatted Output

A terminal window with a dark purple background. The prompt is 'vboxuser@Ubuntu:~/linux_lab\$'. The user enters 'gcc question1c.c -o question1c'. The prompt changes to 'vboxuser@Ubuntu:~/linux_lab\$'. The user enters './question1c'. The program prompts for 'Enter your name: Prathana', 'Enter your age: 22', and 'Enter your height (m): 5.2'. It then displays '--- User Info ---', 'Name: Prathana', 'Age: 22 years', and 'Height: 5.20 m'. The prompt returns to 'vboxuser@Ubuntu:~/linux_lab\$' with a cursor.

```
vboxuser@Ubuntu:~/linux_lab$ gcc question1c.c -o question1c
vboxuser@Ubuntu:~/linux_lab$ ./question1c
Enter your name: Prathana
Enter your age: 22
Enter your height (m): 5.2

--- User Info ---
Name: Prathana
Age: 22 years
Height: 5.20 m
vboxuser@Ubuntu:~/linux_lab$
```

Figure 3: Compilation and Execution of C Program with Formatted Output

Summary:

The C program was compiled and executed successfully using GCC. It accepts the user's name, age, and height as inputs and displays the information in a clearly formatted User Info section.

III. GCC COMPILATION PROCESS

A. Installing GCC on Linux

```
vboxuser@Ubuntu:~/linux_lab$ sudo apt install build-essential
[sudo: authenticate] Password:
build-essential is already the newest version (12.12ubuntu2.26.04.2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 9
vboxuser@Ubuntu:~/linux_lab$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

vboxuser@Ubuntu:~/linux_lab$
```

Figure 4: GCC Installation and Version Verification on Linux

Summary:

Installed the build-essential package using apt and verified the GCC installation by checking its version. The terminal confirms that GCC version **15.2.0** is installed and ready for compiling C programs.

B. Four Stages of Compilation

```
Sep 18 11:38
vboxuser@Ubuntu: ~/linux_lab
~/linux_lab

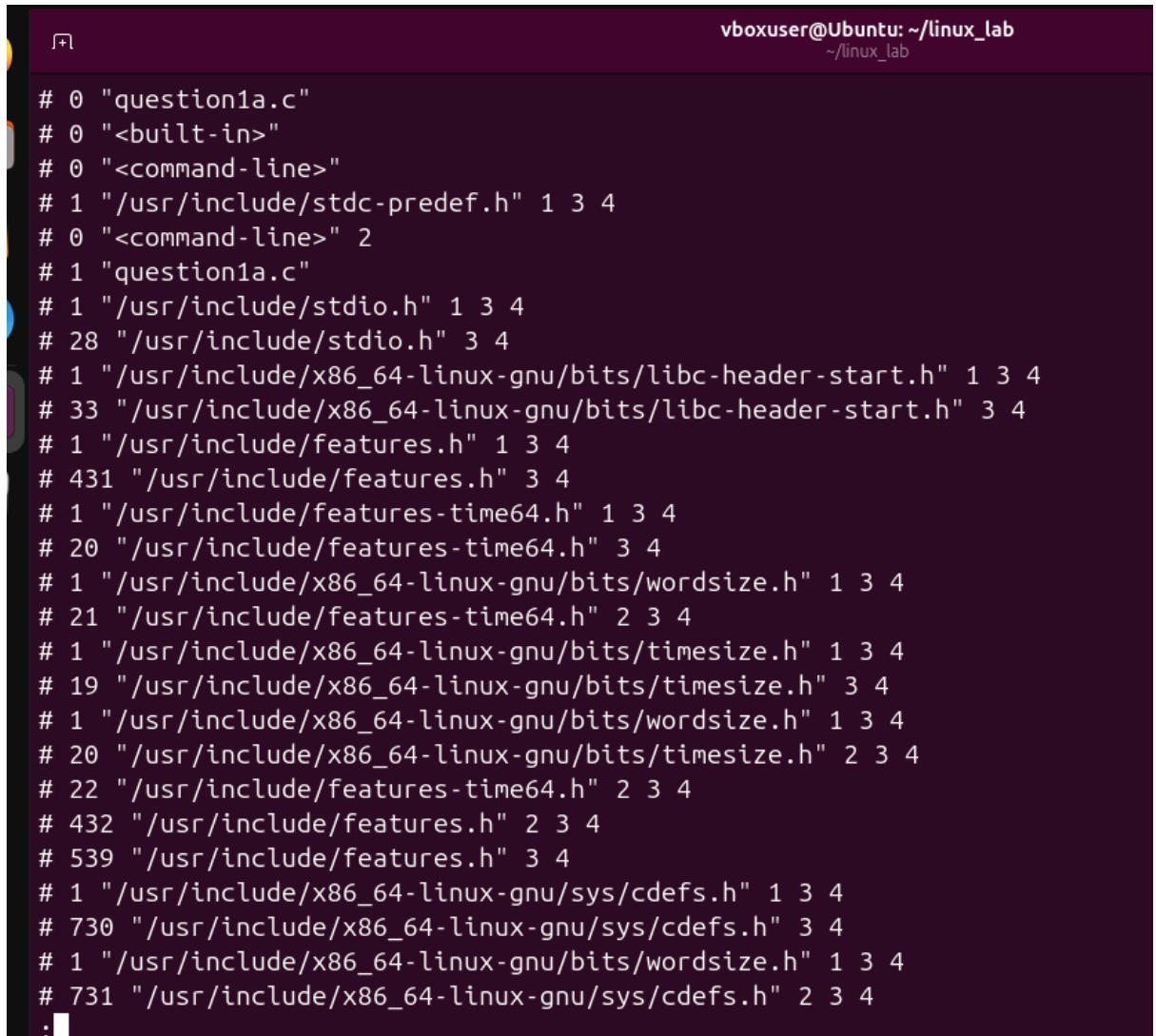
vboxuser@Ubuntu:~/linux_lab$ gcc -E question1a.c -o question1a.i
vboxuser@Ubuntu:~/linux_lab$ ls
function      question1a    question1a.i  question1b.c  question1c.c  user
function.c    question1a.c  question1b    question1c    questiona.i   user.c
vboxuser@Ubuntu:~/linux_lab$
```

Figure 5: Preprocessing Stage of C Compilation

Summary:

The C source file question1c.c was processed using the GCC -E option to generate the preprocessed file question1a.i. The ls command was then used to verify that the generated file was successfully created.

Stage 1: Preprocessing



```
vboxuser@Ubuntu: ~/linux_lab
# 0 "question1a.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "question1a.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 432 "/usr/include/features.h" 2 3 4
# 539 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
:
```

Figure 6: Stage 1 – Preprocessing of C Program

Summary:

The first stage of compilation is preprocessing. The GCC -E option was used to process question1a.c and generate the preprocessed file question1a.i. The ls command was used to verify that the .i file was successfully created.

Stage 2: Compilation to Assembly

```
vboxuser@ubuntu: ~/linux_lab$ cat question1a.c | less
vboxuser@Ubuntu: ~/linux_lab$ gcc -S question1a.c -o question1a.s
vboxuser@Ubuntu: ~/linux_lab$ ls
function  question1a  question1a.i  question1b  question1c  questiona.i  user
function.c  question1a.c  question1a.s  question1b.c  question1c.c
vboxuser@Ubuntu: ~/linux_lab$ cat question1a.s
        .file     "question1a.c"
        .text
        .section   .rodata
.LC0:
        .string   "Hello, World!"
        .text
        .globl    main
        .type      main, @function

main:
.LFB0:
        .cfi_startproc
endbr64
        pushq     %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq      %rsp, %rbp
        .cfi_def_cfa_register 6
        leaq      .LC0(%rip), %rax
        movq      %rax, %rdi
        call      puts@PLT
```

Figure 7: Stage 2 – Compilation of C Program to Assembly

Summary:

The second stage converts the preprocessed C program into assembly language. The GCC -S option was used to generate the assembly file question1a.s. The cat command was then used to view the generated assembly code.

Stage 3: Assembly to Object File

```
vboxuser@Ubuntu: ~/linux_lab
vboxuser@Ubuntu: ~/linux_lab$ gcc -c question1a.s -o question1a.o
vboxuser@Ubuntu: ~/linux_lab$ file question1a.o
question1a.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
vboxuser@Ubuntu: ~/linux_lab$
```

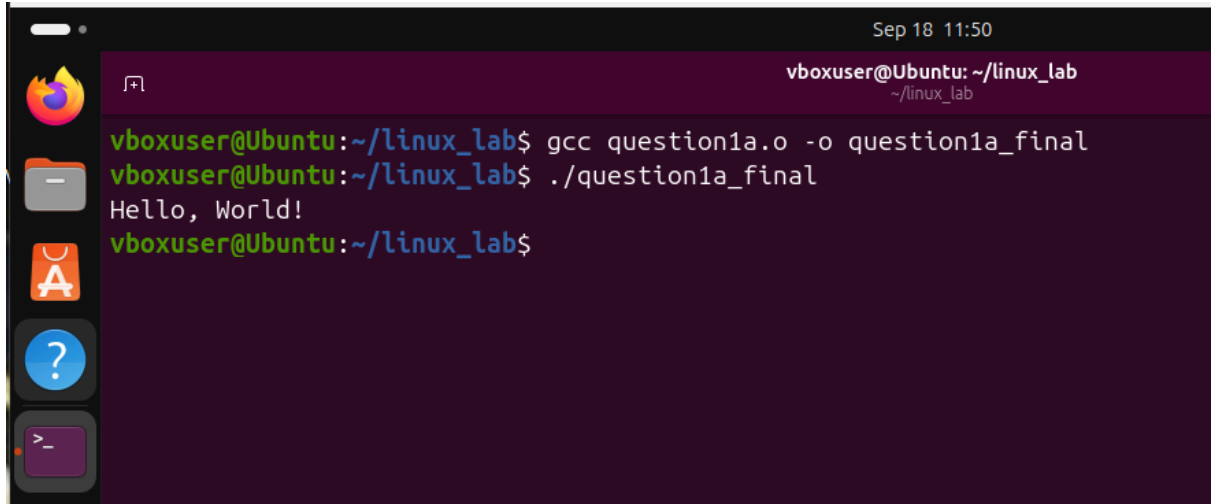
Figure 8: Stage 3 – Conversion of Assembly Code to Object File

Summary:

The third stage converts the assembly code into an object file. The GCC -c option

was used to create question1a.o. The file command verified that the generated file is a valid 64-bit ELF relocatable object file.

Stage 4: Linking



```
Sep 18 11:50
vboxuser@Ubuntu: ~/linux_lab
vboxuser@Ubuntu:~/linux_lab$ gcc question1a.o -o question1a_final
vboxuser@Ubuntu:~/linux_lab$ ./question1a_final
Hello, World!
vboxuser@Ubuntu:~/linux_lab$
```

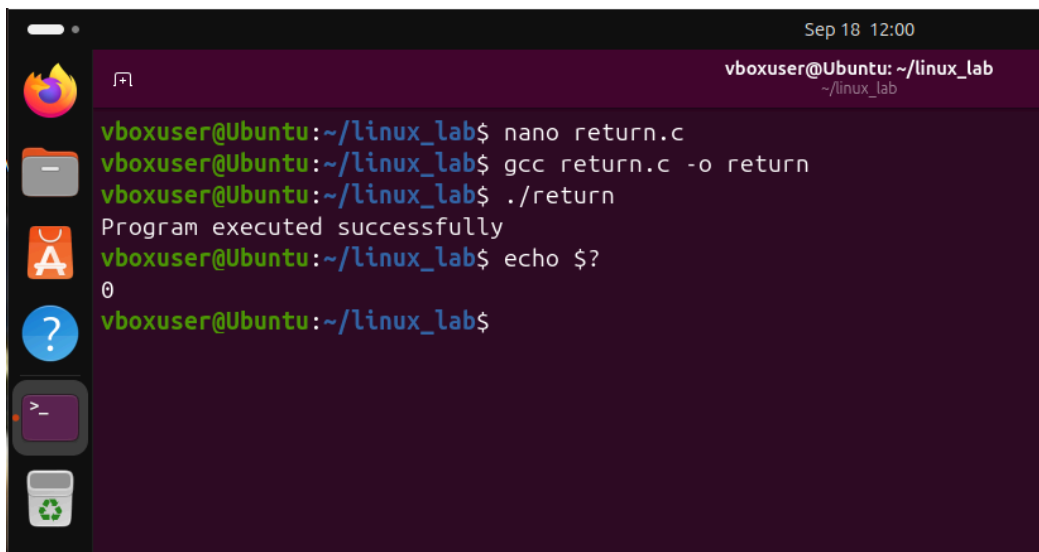
Figure 9: Stage 4 – Linking Object File to Create Executable

Summary:

The fourth and final stage is linking. The object file question1a.o was linked using GCC to create the executable file question1a_final. The executable was then run successfully, displaying the expected “Hello, World!” output.

IV. RETURN VALUES AND EXIST STATUS

A. Understanding Return Values



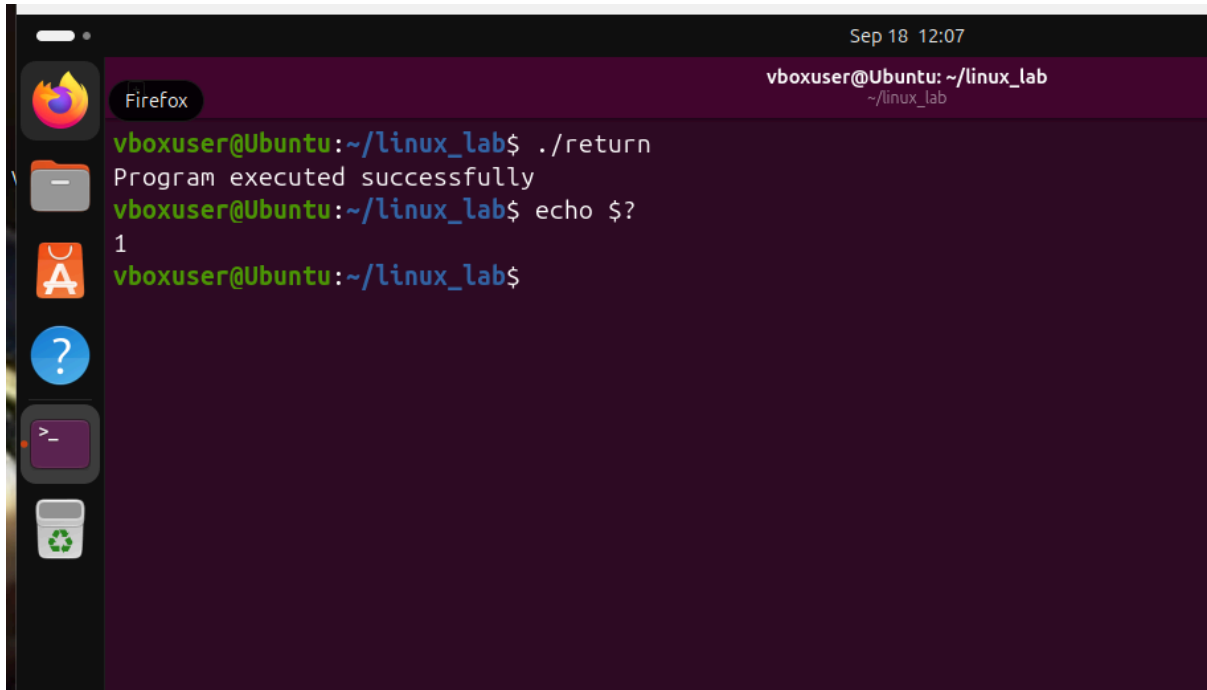
```
Sep 18 12:00
vboxuser@Ubuntu: ~/linux_lab
vboxuser@Ubuntu:~/linux_lab$ nano return.c
vboxuser@Ubuntu:~/linux_lab$ gcc return.c -o return
vboxuser@Ubuntu:~/linux_lab$ ./return
Program executed successfully
vboxuser@Ubuntu:~/linux_lab$ echo $?
0
vboxuser@Ubuntu:~/linux_lab$
```

Figure 10: C Program Execution and Return Value Verification

Summary:

The C program was compiled and executed successfully. The echo \$? command was used to check the program's return value, which displayed **0**, indicating successful execution.

B. Checking Exit Status

A screenshot of a Linux terminal window. The window title bar shows 'vboxuser@Ubuntu: ~/linux_lab' and the date 'Sep 18 12:07'. The terminal has a dark purple background. The prompt is 'vboxuser@Ubuntu:~/linux_lab\$'. The first command entered is './return', followed by the output 'Program executed successfully'. The second command entered is 'echo \$?', followed by the output '1'. The prompt returns to 'vboxuser@Ubuntu:~/linux_lab\$'. On the left side of the terminal, there is a vertical dock with icons for Firefox, a file manager, an application store, a help icon, and a trash can.

```
vboxuser@Ubuntu:~/linux_lab$ ./return
Program executed successfully
vboxuser@Ubuntu:~/linux_lab$ echo $?
1
vboxuser@Ubuntu:~/linux_lab$
```

Figure 11: Checking Program Exit Status Using echo \$?

Summary:

The previously executed program was checked using the echo \$? command to verify its exit status. The terminal shows the program completed successfully, demonstrating how Linux reports the status of the most recently executed command.

V. KNOWLEDGE CHECK QUESTIONS

A. Question 1

Q: What is the purpose of the #include directive in C programming?
Provide an example of its usage .

- The #include directive is used to include a header file in a C program. Header files contain predefined functions and declarations that can be used in the program.

Example:

```
#include <stdio.h>
```

Here, `stdio.h` is a standard input/output header file. It allows us to use functions such as `printf()` and `scanf()`.

B. Question 2

Q: Explain the difference between the four stages of compilation. What file extension is produced at each stage?

Stage	Description	File Extension
Preprocessing	Processes preprocessor directives such as <code>#include</code> and expand macros.	.i
Compilation	Converts the preprocessed C source code into assembly language.	.s
Assembly	Converts assembly language into machine/object code.	.o
Linking	Combines object files and required libraries to produce the final executable program.	Executable

C. Question 3

Q: What does a return value of 0 from the `main()` function indicate to the operating system? What about a non-zero return value?

- A return value of 0 from the `main()` function indicates that the program completed successfully.

A non-zero return value generally indicates that the program terminated with an error or an unsuccessful status. Different non-zero values can be used to represent different types of errors.

Example:

```
return 0; // Successful execution
return 1; // Unsuccessful execution
```

D. Question 4

Q: Write a C program that:

- Declares an integer variable
- Asks the user to input a value
- Displays the entered value back to the user

Answer:

```
#include <stdio.h>

int main() {
    int number;

    printf("Enter an integer: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);

    return 0;
}
```

E. Question 5

Q: What GCC command would you use to:

1) Only preprocess a file without compiling it?

```
gcc -E hello.c -o hello.i
```

The -E option performs only preprocessing and produces a .i file.

2) Generate assembly code but not assemble it?

```
gcc -S hello.c -o hello.s
```

The -S option generates assembly code and produces a .s file.3) Create an object file without linking?

```
gcc -c hello.c -o hello.o
```

F . Question 6

Q: Explain why C is called a "compiled language" and how this differs from interpreted languages.

C is called a compiled language because its source code is translated into machine code by a compiler before the program is executed. GCC is commonly used to compile C programs.

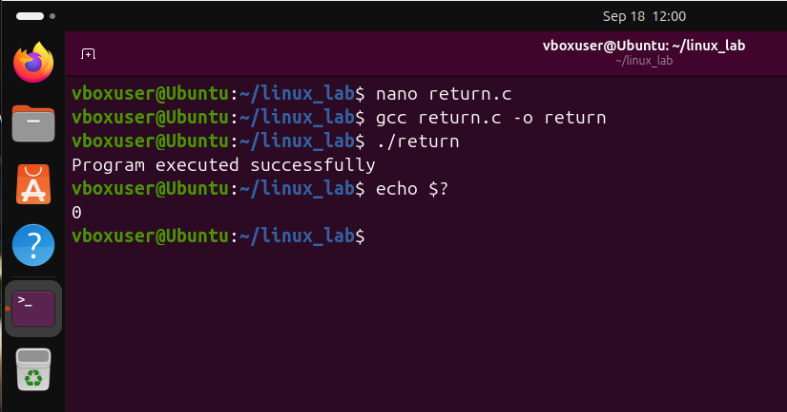
In an interpreted language, the source code is generally executed through an interpreter at runtime rather than first being converted into a standalone executable in the same way as a typical C program.

For example, a C program can be compiled using:

```
gcc program.c -o program
```

VI . LAB TASKS COMPLETED

1) Task 1: Compiled hello.c with return 0 and verified exit status using echo \$?

A terminal window with a dark purple background and light green text. The window title is "vboxuser@Ubuntu: ~/linux_lab". The terminal shows the following commands and output:

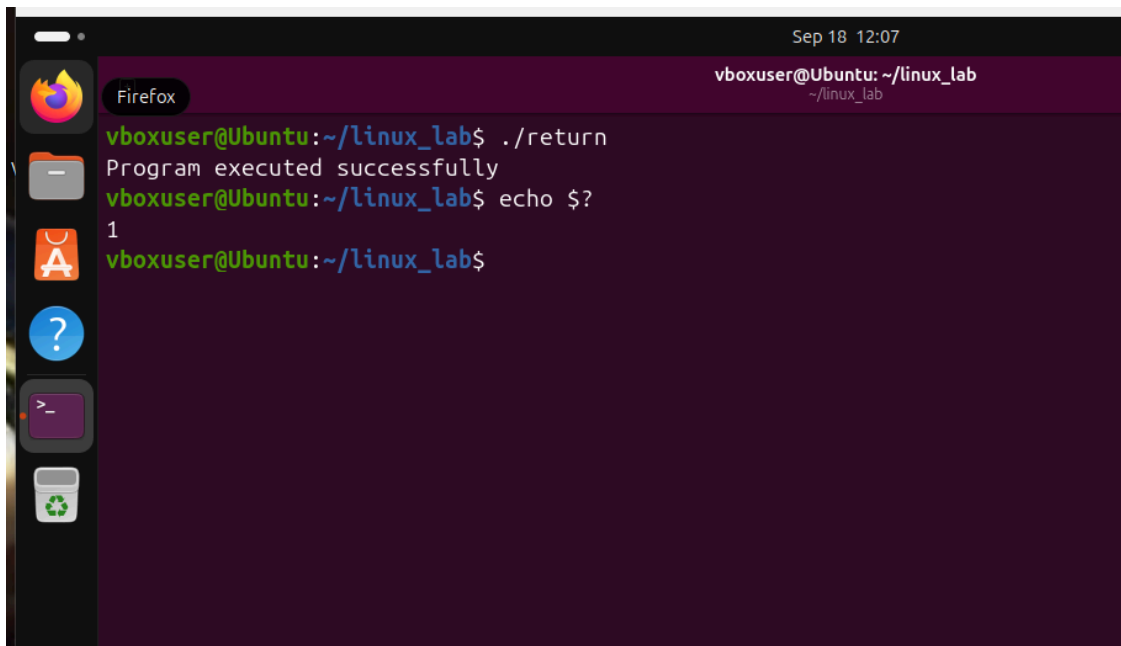
```
vboxuser@Ubuntu:~/linux_lab$ nano return.c
vboxuser@Ubuntu:~/linux_lab$ gcc return.c -o return
vboxuser@Ubuntu:~/linux_lab$ ./return
Program executed successfully
vboxuser@Ubuntu:~/linux_lab$ echo $?
0
vboxuser@Ubuntu:~/linux_lab$
```

Figure 12: Task 1 – Program Execution with Return Value 0

Summary:

The C program was compiled and executed successfully using GCC. The `echo $?` command displayed 0, confirming that the program terminated successfully with a normal exit status.

2) Task 2: Modified program to return 1 and observed different exit status.

A terminal window titled 'vboxuser@Ubuntu: ~/linux_lab' with a timestamp of 'Sep 18 12:07'. The terminal shows the following commands and output:

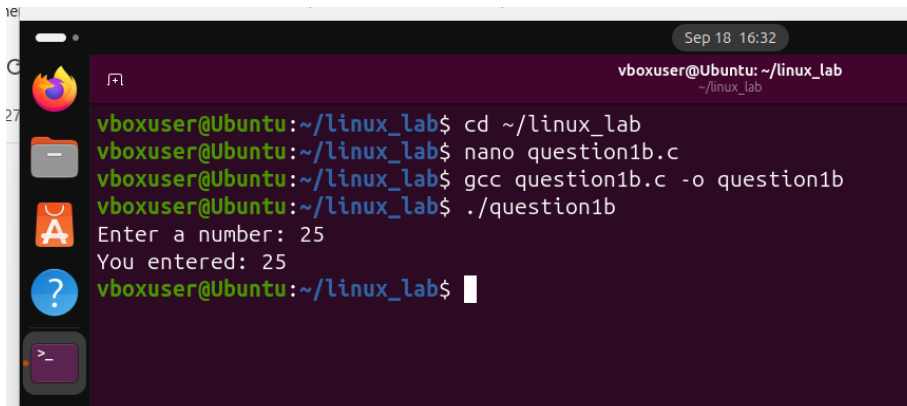
```
vboxuser@Ubuntu:~/linux_lab$ ./return
Program executed successfully
vboxuser@Ubuntu:~/linux_lab$ echo $?
1
vboxuser@Ubuntu:~/linux_lab$
```

Figure 13: Task 2 – Program Execution with Return Value 1

Summary:

The program was modified to return 1 instead of 0 and executed again. Using `echo $?` displayed 1, demonstrating how a non-zero return value indicates a different exit status in Linux.

3) Task 3: Created program accepting user input and displaying it

A terminal window titled 'vboxuser@Ubuntu: ~/linux_lab' with a timestamp of 'Sep 18 16:32'. The terminal shows the following commands and output:

```
vboxuser@Ubuntu:~/linux_lab$ cd ~/linux_lab
vboxuser@Ubuntu:~/linux_lab$ nano question1b.c
vboxuser@Ubuntu:~/linux_lab$ gcc question1b.c -o question1b
vboxuser@Ubuntu:~/linux_lab$ ./question1b
Enter a number: 25
You entered: 25
vboxuser@Ubuntu:~/linux_lab$
```

Figure 14: Task 3 – C Program Accepting and Displaying User Input

Summary:

A C program was created to accept a number from the user. The program was compiled and executed successfully, with 25 entered as the input and "You entered: 25" displayed as the output.

4) Task 4: Executed all four compilation stages separately and documented output files

Stage 1-

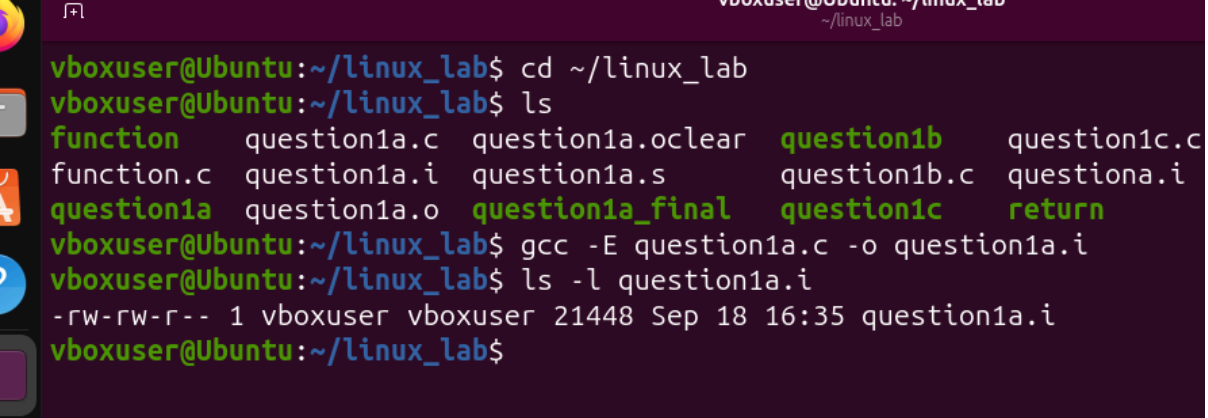
A terminal window with a dark purple background. The prompt is 'vboxuser@Ubuntu: ~/linux_lab'. The user enters 'cd ~/linux_lab'. The prompt changes to 'vboxuser@Ubuntu: ~/linux_lab\$'. The user enters 'ls', showing a directory listing of files including 'function', 'question1a.c', 'question1a.oclear', 'question1b', 'question1c.c', 'function.c', 'question1a.i', 'question1a.s', 'question1b.c', 'questiona.i', 'question1a', 'question1a.o', 'question1a_final', 'question1c', and 'return'. The user then enters 'gcc -E question1a.c -o question1a.i'. The prompt changes to 'vboxuser@Ubuntu: ~/linux_lab\$'. The user enters 'ls -l question1a.i', showing the file permissions '-rw-rw-r-- 1 vboxuser vboxuser 21448 Sep 18 16:35 question1a.i'. The prompt changes to 'vboxuser@Ubuntu: ~/linux_lab\$'.

Figure 15: Task 4 – Stage 1 Preprocessing and Generated Output File

Summary:

The first compilation stage was performed using GCC with the -E option. The preprocessed file question1a.i was generated successfully, and the ls -l command was used to verify the output file.

Stage 2

```
15 -l question1a.s
~/linux_lab

vboxuser@Ubuntu:~/linux_lab$ gcc -E question1a.c -o question1a.i
vboxuser@Ubuntu:~/linux_lab$ ls -l question1a.i
-rw-rw-r-- 1 vboxuser vboxuser 21448 Sep 18 16:35 question1a.i
vboxuser@Ubuntu:~/linux_lab$ gcc -S question1a.c -o question1a.s
vboxuser@Ubuntu:~/linux_lab$ ls -l
total 216
-rwxrwxr-x 1 vboxuser vboxuser 15984 Sep 16 12:13 function
-rw-rw-r-- 1 vboxuser vboxuser 151 Sep 16 12:13 function.c
-rwxrwxr-x 1 vboxuser vboxuser 15960 Sep 18 10:17 question1a
-rw-rw-r-- 1 vboxuser vboxuser 73 Sep 18 10:16 question1a.c
-rw-rw-r-- 1 vboxuser vboxuser 21448 Sep 18 16:35 question1a.i
-rw-rw-r-- 1 vboxuser vboxuser 1496 Sep 18 11:48 question1a.o
-rw-rw-r-- 1 vboxuser vboxuser 1496 Sep 18 11:49 question1a.o.clear
-rw-rw-r-- 1 vboxuser vboxuser 666 Sep 18 16:37 question1a.s
-rwxrwxr-x 1 vboxuser vboxuser 15960 Sep 18 11:50 question1a_final
-rwxrwxr-x 1 vboxuser vboxuser 16064 Sep 18 16:30 question1b
-rw-rw-r-- 1 vboxuser vboxuser 149 Sep 18 16:30 question1b.c
-rwxrwxr-x 1 vboxuser vboxuser 16104 Sep 18 11:31 question1c
-rw-rw-r-- 1 vboxuser vboxuser 384 Sep 18 11:30 question1c.c
-rw-rw-r-- 1 vboxuser vboxuser 21448 Sep 18 11:35 questiona.i
-rwxrwxr-x 1 vboxuser vboxuser 15952 Sep 18 12:04 return
-rw-rw-r-- 1 vboxuser vboxuser 112 Sep 18 12:02 return.c
-rwxrwxr-x 1 vboxuser vboxuser 15952 Sep 18 12:14 task5
-rw-rw-r-- 1 vboxuser vboxuser 90 Sep 18 12:14 task5.c
-rwxrwxr-x 1 vboxuser vboxuser 16056 Sep 16 12:00 user
-rw-rw-r-- 1 vboxuser vboxuser 155 Sep 16 11:59 user.c
vboxuser@Ubuntu:~/linux_lab$
```

Figure 16: Task 4 – Stage 2 Compilation to Assembly

Summary:

The second compilation stage was performed using the GCC -S option. The preprocessed C program was converted into assembly code, and the generated question1a.s file was verified using the ls command.

Stage 3

```
vboxuser@Ubuntu: ~/linux_lab
~/linux_lab

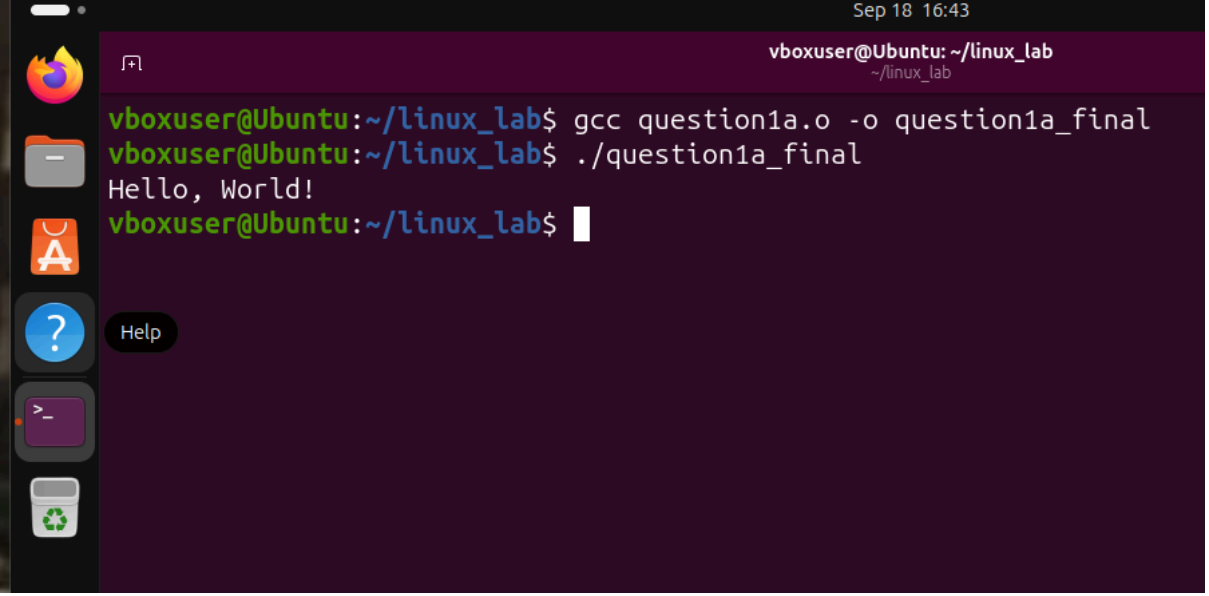
vboxuser@Ubuntu:~/linux_lab$ gcc -c question1a.s -o question1a.o
vboxuser@Ubuntu:~/linux_lab$ ls -l question1a.o
-rw-rw-r-- 1 vboxuser vboxuser 1496 Sep 18 16:40 question1a.o
vboxuser@Ubuntu:~/linux_lab$
```

Figure 17: Task 4 – Stage 3 Assembly to Object File

Summary:

The third compilation stage was performed using the GCC -c option. The assembly file was converted into the object file question1a.o, and the ls -l command confirmed that the object file was successfully generated.

Stage 4



```

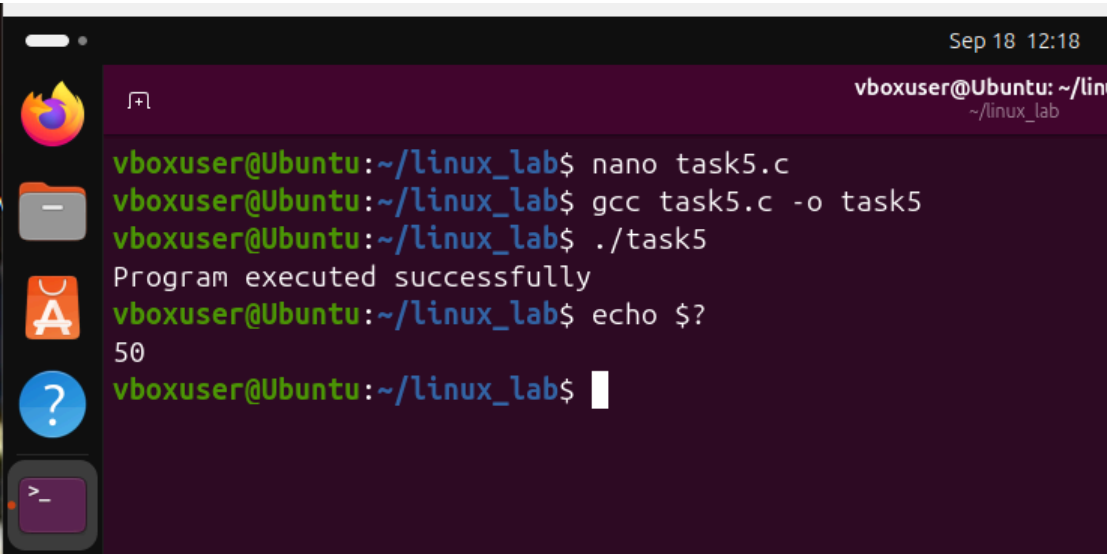
vboxuser@Ubuntu: ~/linux_lab
vboxuser@Ubuntu:~/linux_lab$ gcc question1a.o -o question1a_final
vboxuser@Ubuntu:~/linux_lab$ ./question1a_final
Hello, World!
vboxuser@Ubuntu:~/linux_lab$
```

Figure 18: Task 4 – Stage 4 Linking and Execution

Summary:

The fourth compilation stage linked the object file question1a.o to create the executable question1a_final. The executable was then run successfully, producing the expected “Hello, World!” output.

5) Task 5: Compiled and ran program returning integer value 50



```

vboxuser@Ubuntu: ~/linux_lab
vboxuser@Ubuntu:~/linux_lab$ nano task5.c
vboxuser@Ubuntu:~/linux_lab$ gcc task5.c -o task5
vboxuser@Ubuntu:~/linux_lab$ ./task5
Program executed successfully
vboxuser@Ubuntu:~/linux_lab$ echo $?
50
vboxuser@Ubuntu:~/linux_lab$
```

Figure 19: Task 5 – Program Execution with Return Value 50

Summary:

A C program was created and compiled using GCC with a return value of 50. After execution, the echo \$? command displayed 50, confirming that the program returned the specified integer value as its exit status.